LEARN BEING FORWARD

Programming — Python Programming

OOP in Python

COMPLETE LEARNING GUIDE

What You'll Learn

Object-oriented programming in Python bundles data and the behavior that acts on it into classes, and while Python's OOP is intentionally less ceremonious than Java's or C++'s (no mandatory access modifiers, no explicit interfaces), the core ideas — encapsulation, inheritance, and polymorphism — work the same way underneath. This sub-module covers class and instance fundamentals, the instance-vs-class attribute distinction that regularly confuses newcomers, inheritance and super(), Python's convention-based (rather than enforced) approach to encapsulation via properties, the dunder methods that let custom objects integrate with built-in functions like print() and len(), duck-typing-driven polymorphism, formal abstract base classes for when an explicit contract is actually needed, and the dataclasses module that eliminates most of the boilerplate in simple data-holding classes.

1. Classes, Objects, and __init__

A class is a blueprint for objects; __init__ is the initializer method Python calls automatically right after a new instance is created, and is where instance attributes are typically set via self. self is the instance itself, passed automatically as the first argument to every instance method — it is a naming convention (not a keyword), though changing it is strongly discouraged since every Python developer expects it. Creating an object is just calling the class like a function: ClassName(args).

PYTHON

```python
class Dog:
    def __init__(self, name: str, breed: str):
        self.name = name
        self.breed = breed

    def bark(self) -> str:
        return f"{self.name} says Woof!"

rex = Dog("Rex", "Labrador")
print(rex.bark())  # Rex says Woof!
```

2. Instance Attributes vs. Class Attributes

An attribute defined inside __init__ via self.x = ... belongs to that specific instance; an attribute defined directly in the class body (outside any method) is a class attribute, shared by every instance unless an instance later sets its own attribute of the same name, which shadows the class attribute for that instance only. This distinction matters most with mutable class attributes: a mutable default like a list, assigned at the class level, is shared across all instances and mutating it through one instance silently affects every other instance too.

- A mutable class attribute (e.g. `tags = []` in the class body) is shared by every instance — mutate it via self.tags.append(...) and ALL instances see the change.
- Class attributes are accessible via either the class or an instance (Dog.species or rex.species); instance attributes only exist on that specific instance.

```python
class Dog:
    species = "Canis familiaris"  # class attribute — shared by all instances

    def __init__(self, name: str):
        self.name = name          # instance attribute — unique per instance

a = Dog("Rex")
b = Dog("Fido")
print(a.species, b.species)  # Canis familiaris Canis familiaris

Dog.species = "Canis lupus familiaris"  # changes it for every instance
print(a.species)  # Canis lupus familiaris
```

3. Inheritance and super()

A class can inherit from another by listing it in parentheses (class Puppy(Dog):), gaining all of the parent's methods and attributes while being free to override any of them. super() returns a proxy to the parent class, letting a subclass's overridden method call the parent's original implementation instead of fully replacing it — this is the standard way to extend __init__ in a subclass without duplicating the parent's setup logic.

```python
class Animal:
    def __init__(self, name: str):
        self.name = name

    def speak(self) -> str:
        return f"{self.name} makes a sound"

class Dog(Animal):
    def __init__(self, name: str, breed: str):
        super().__init__(name)   # reuse Animal's init instead of repeating it
        self.breed = breed

    def speak(self) -> str:      # override
        return f"{self.name} barks"
```

```
print(Dog("Rex", "Lab").speak())  # Rex barks
```

4. Encapsulation: Naming Conventions and Properties

Python has no true private attributes enforced by the language — a single leading underscore (_name) is a convention meaning 'internal, don't touch from outside,' and a double leading underscore (__name) triggers name-mangling (rewritten internally as _ClassName__name) to avoid accidental clashes in subclasses, but neither is truly inaccessible. The @property decorator is the idiomatic way to expose a method as if it were a plain attribute, which lets validation or computed logic run on get/set without changing the calling code's syntax from attribute access to method calls.

- A single underscore prefix (_x) is purely a convention signaling 'internal use' — Python does not prevent external access.
- @property with a matching @x.setter lets `obj.x = value` run validation logic transparently, instead of requiring `obj.set_x(value)`.

```python
class Account:
    def __init__(self, balance: float):
        self._balance = balance

    @property
    def balance(self) -> float:
        return self._balance

    @balance.setter
    def balance(self, value: float):
        if value < 0:
            raise ValueError("Balance cannot be negative")
        self._balance = value

acc = Account(100)
acc.balance = 50       # runs the setter's validation
print(acc.balance)     # 50 — reads like a plain attribute, runs like a method
```

5. Dunder Methods: Integrating With Built-ins

'Dunder' (double-underscore) methods let a custom class plug into Python's built-in syntax and functions. __str__ controls what str(obj) and print(obj) show (a human-readable form); __repr__ controls the unambiguous developer-facing representation shown in a REPL or debugger (and is used as a fallback if __str__ isn't defined); __eq__ controls == comparisons; __len__ lets len(obj) work. Defining these turns a custom class into something that behaves like a 'real' Python type throughout the standard library, rather than needing special-cased handling everywhere it's used.

- If __eq__ is defined but __hash__ is not explicitly kept, instances become unhashable by default and can't be used in sets or as dict keys.

• A good convention: __repr__ should ideally look like valid Python code that could recreate the object.

```python
class Point:
    def __init__(self, x: float, y: float):
        self.x, self.y = x, y

    def __str__(self) -> str:
        return f"({self.x}, {self.y})"

    def __repr__(self) -> str:
        return f"Point(x={self.x}, y={self.y})"

    def __eq__(self, other) -> bool:
        return isinstance(other, Point) and self.x == other.x and self.y == other.y

p1, p2 = Point(1, 2), Point(1, 2)
print(p1)          # (1, 2)          — via __str__
print(p1 == p2)    # True            — via __eq__
```

6. Polymorphism and Duck Typing

Polymorphism means code written against one interface works correctly with many different types that implement it — in Python this is almost always achieved through duck typing ('if it walks like a duck and quacks like a duck, treat it as a duck') rather than requiring a shared formal base class: a function calling obj.speak() works with any object that has a .speak() method, regardless of its class hierarchy. This makes Python code highly flexible, but also means a typo'd or missing method surfaces only at runtime (an AttributeError), not at compile time, since there's no compiler enforcing the contract in advance.

```python
class Dog:
    def speak(self): return "Woof"

class Cat:
    def speak(self): return "Meow"

class Duck:
    def speak(self): return "Quack"

for animal in [Dog(), Cat(), Duck()]:
    print(animal.speak())  # works for all three — no shared base class required
```

7. Abstract Base Classes

When duck typing's implicit contract isn't strict enough — when you genuinely want to force every subclass to implement certain methods, and fail loudly at instantiation time if they don't — the abc module's ABC base class and @abstractmethod decorator provide that. A class inheriting from ABC with one or more @abstractmethod-decorated methods cannot be instantiated directly until every abstract method is overridden in a concrete subclass, giving

Python something close to a formal interface when a codebase's design genuinely calls for one.

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        ...

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2

# Shape()  # TypeError: Can't instantiate abstract class Shape
print(Circle(2).area())  # 12.56636
```

8. Dataclasses: Removing Boilerplate

Simple classes that mostly just hold data traditionally require a lot of repetitive boilerplate: an __init__ assigning every field, a __repr__ for readable printing, and __eq__ for comparing instances by value. The @dataclass decorator (from the dataclasses module, standard since 3.7) generates all three automatically from type-annotated class-level field declarations, which keeps such classes short while still behaving like full-featured, comparison-aware objects.

- @dataclass(frozen=True) makes instances immutable after construction, which also makes them hashable by default — useful for value-like objects.

```python
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p1 = Point(1, 2)
p2 = Point(1, 2)
print(p1)          # Point(x=1, y=2)  — __repr__ generated automatically
print(p1 == p2)     # True             — __eq__ generated automatically
```

Common Pitfalls

- Using a mutable class-level attribute (like a list) as if it were per-instance state, then being surprised that every instance shares the same mutations.
- Forgetting super().__init__(...) in a subclass's __init__, leaving the parent class's setup logic never run.
- Believing a single or double leading underscore makes an attribute truly private — Python only discourages external access by convention, it does not enforce it.
- Defining __eq__ without considering that it removes the default __hash__, making instances suddenly unusable as dict keys or in sets.
- Writing a full class with just __init__, a handful of fields, and manual __repr__/__eq__ when @dataclass would generate all of that automatically.
- Reaching for abc.ABC and @abstractmethod reflexively when plain duck typing is sufficient for the use case — ABCs add ceremony that isn't always needed in Python.

Key Takeaways

- self is the instance, passed automatically to every instance method; __init__ runs automatically right after object creation to set up instance state.
- Class attributes are shared across all instances unless shadowed by an instance attribute of the same name — be careful with mutable class attributes.
- super() calls the parent class's version of a method, most commonly used to extend (not replace) __init__ in a subclass.
- Dunder methods (__str__, __repr__, __eq__, __len__, ...) are how a custom class plugs into Python's built-in syntax and functions.
- Python favors duck typing over formal interfaces — but abc.ABC with @abstractmethod is available when a strict contract is genuinely needed.
- @dataclass auto-generates __init__, __repr__, and __eq__ for classes that are primarily data containers, cutting significant boilerplate.

Learn More

Python Docs — Classes
https://docs.python.org/3/tutorial/classes.html

Python Docs — dataclasses
https://docs.python.org/3/library/dataclasses.html

Python Docs — abc: Abstract Base Classes
https://docs.python.org/3/library/abc.html